Imperial College London

Lecture 6

Sequential Circuits

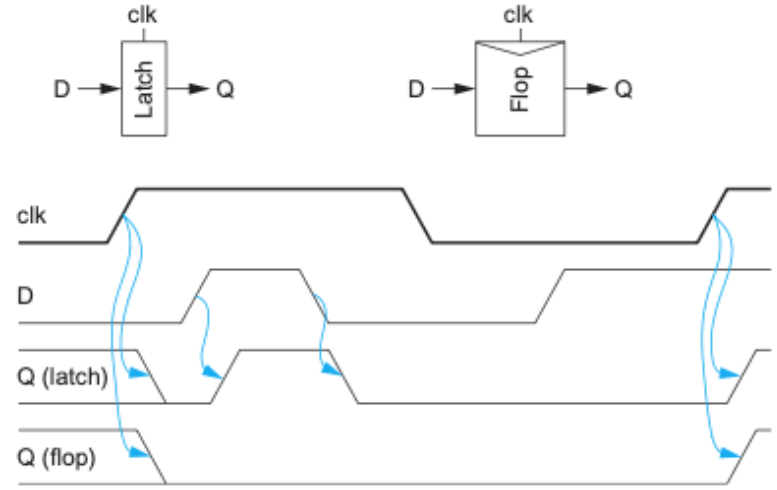
Prof Peter YK Cheung

Department of Electrical & Electronic Engineering

URL: https://elec50010-iac.github.io/course-website/
email: p.cheung@imperial.ac.uk

In this lecture, we will examine the different way that combinational logic gates are implemented in full-custom VLSI circuits.

Latches vs Flip-flops



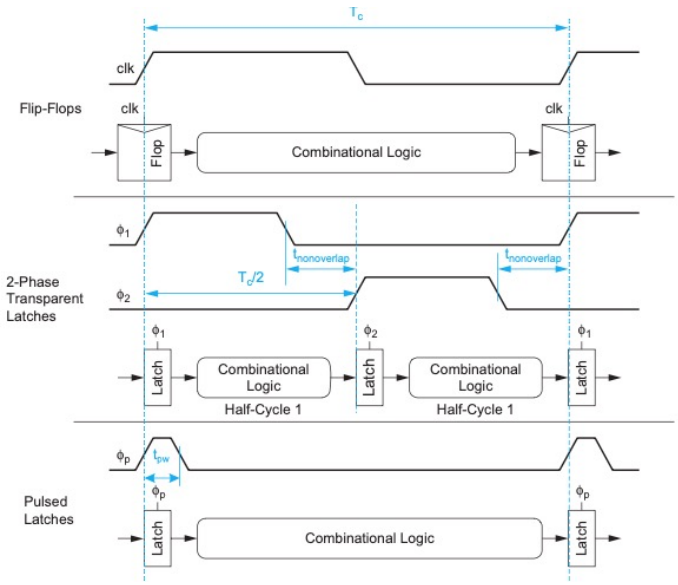
W&H p376

You should be very familiar with flip-flops and latches from modules taught in earlier years.

In Years 2 and 3, you were mostly implementing digital hardware using FPGAs, where only D-FF are used, not latches.

However, in full-custom VLSI design, both D-FF and latches are being used. A latch differs from a D-FF in that it is NOT edge triggered. Instead, the output takes on the value of the input (transparent) as long as the clock signal is active (usually high). This is shown in the timing diagram above.

Clocking Styles



W&H p377-378

There are three common clocking styles used with digital VLSI sequential circuits:

1. Edge triggered clocking;

2. Two-phase non-overlapping clocking;

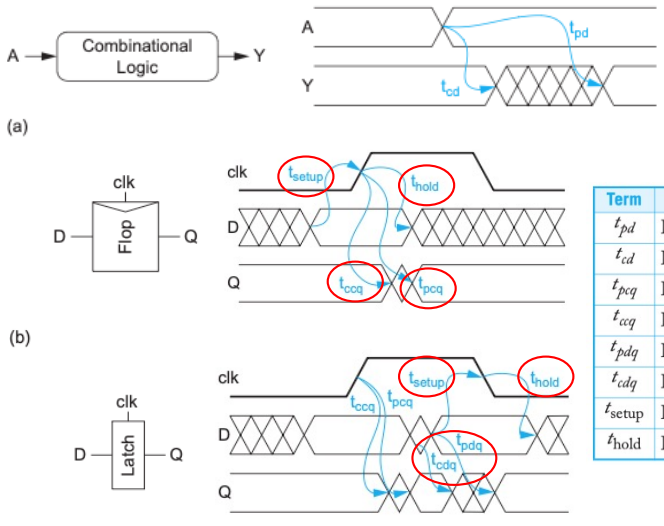
3. Pulse clocking;

A D-FF pipeline (top diagram) responses to the rising edge of the clock signal. The combinational logic in between the two pipeline flops perform the logic evaluation over the clock period T_c.

In VLSI design, using D-FF alone would reduce the opportunity for better timing and area optimization. Therefore, many full-custom digital VLSI circuits use a two-phase transparent latch circuit as shown in the middle diagram above. The two clock signals ϕ_1 and ϕ_2 do not overlap. That is, there is a period in between the two high states where both clock signals are LOW. This ensures that the pipelined signal stored in the latch will not race forward by mistake. Such a clocking method allows logic operations to be perform on BOTH phases of the clock signal.

Finally, one could use a short pulse to drive a clock input of a transparent latch between each combinational logic block. This style of clocking is common among recent high performance processor designs. However, such designs must be carefully simulated to ensure no timing violation under all PVT conditions.

Timing considerations for sequential circuits



Term	Name
t_{pd}	Logic Propagation Delay
t_{cd}	Logic Contamination Delay
t_{pcq}	Latch/Flop Clock-to-Q Propagation Delay
t_{ccq}	Latch/Flop Clock-to-Q Contamination Delay
t_{pdq}	Latch D-to-Q Propagation Delay
t_{cdq}	Latch D-to-Q Contamination Delay
t_{setup}	Latch/Flop Setup Time
t_{hold}	Latch/Flop Hold Time

W&H p378-379

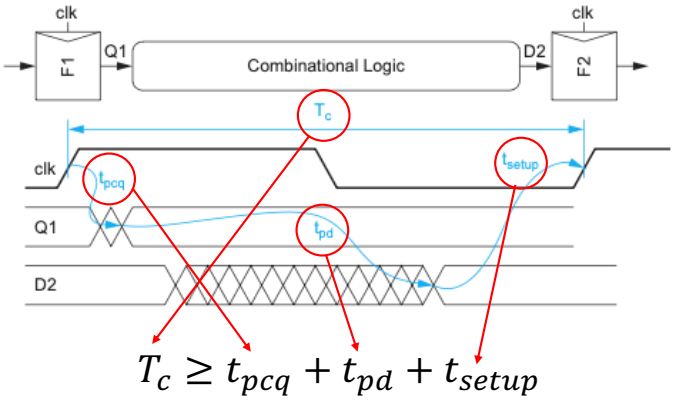
The table above defines the delays and timing constraints of the combinational logic and sequencing elements. These delays may differ significantly for rising and falling transitions and can be distinguished with an r or f suffix.

Figure (a) shows the response of combinational logic to the input A changing from one arbitrary value to another. The output Y cannot change instantaneously. Contamination delay t_{cd} (new term) is the time between input changing and output changing, but not necessarily showing the correct value (contaminated). Propagation delay t_{pd} is the time it takes for the output to settle to a final value.

Figure (b) shows the response of a flip-flop. You are familiar with the setup and hold times (t_{setup} and t_{hold}). Data MUST BE stable at least t_{setup} before the rising clock edge, and must hold its value for t_{hold} or longer. The output begins to change after a clock-to-Q contamination delay t_{ccq} and completely settles after a clock-to-Q propagation delay t_{pcq}.

Figure (c) shows the response of a latch. Unlike for the case of D-FF in (b), the input D must set up and hold around the falling edge (not rising edge) of the clock. Further, since the latch is "transparent", its output will change if input changes while the clock signal is high. The latch has two further timing constraints: the input D-to-Q contamination and propagation delays: t_{cdq} and t_{pdq}.

Flip-flop Maximum Delay Constraint



$$T_c \geq t_{pcq} + t_{pd} + t_{setup}$$

$$t_{pd} \leq T_c - \underbrace{(t_{setup} + t_{pcq})}_{\text{sequencing overhead}}$$

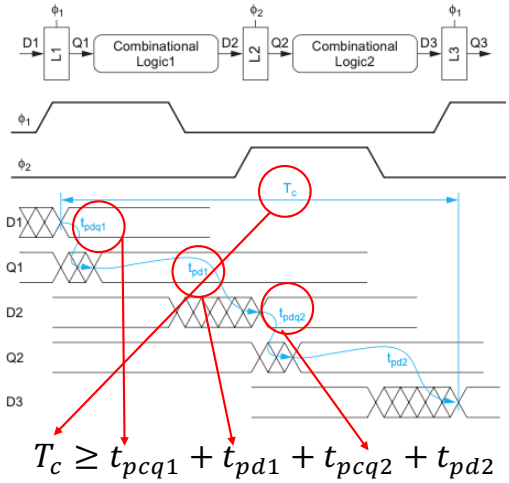
W&H p380

In the case of the D-FF, the cycle time T_c must be at least as long as the sum of the flip-flop setup time t_{setup}, the propagation delay from clock-to-Q t_{pcq} and the delay of the combinational logic t_{pd}.

It is common that the desired operating frequency is known, and therefore T_c is defined. This provides the constraints on how much logic circuit can be introduced between two pipeline stages.

The quantity $(t_{setup}+t_{pcq})$ is the delay overhead incurred by introducing sequential circuit (i.e. the D-FF).

Latch Maximum Delay Constraint



$$T_c \geq t_{pcq1} + t_{pd1} + t_{pcq2} + t_{pd2}$$

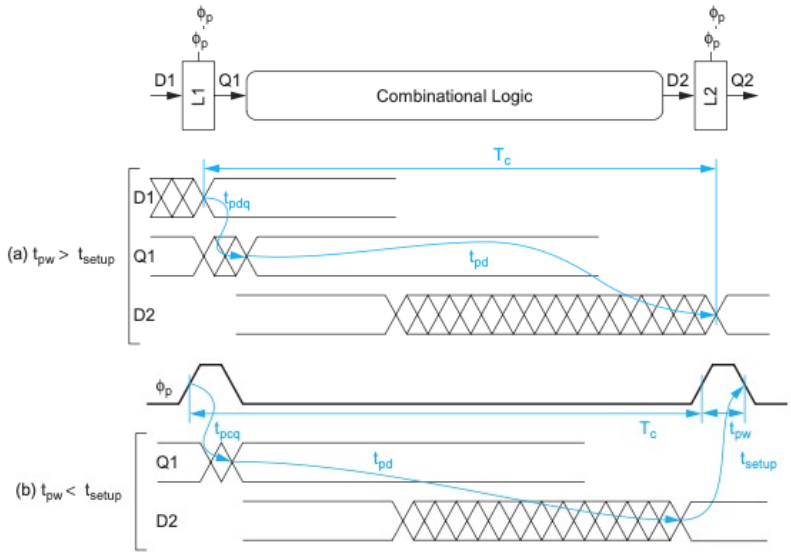
$$t_{pd} \leq T_c - (2t_{pcq})$$

sequencing overhead

W&H p382

The maximum delay constraint for a 2-phase latch sequential circuit is shown above. Assuming that identical latches are used, the sequencing overhead is just twice that of the propagation delay from clock to Q (i.e. $2t_{pcq}$).

Pulsed Latch Maximum Delay Constraint



W&H p382-383

The max-delay constraint for pulsed latches is similar to that of two-phase latches except that only one latch is in the critical path (see (a) above).

However, if the pulse is narrower than the setup time, the data must set up before the pulse rises, as shown in Figure 1 (b).

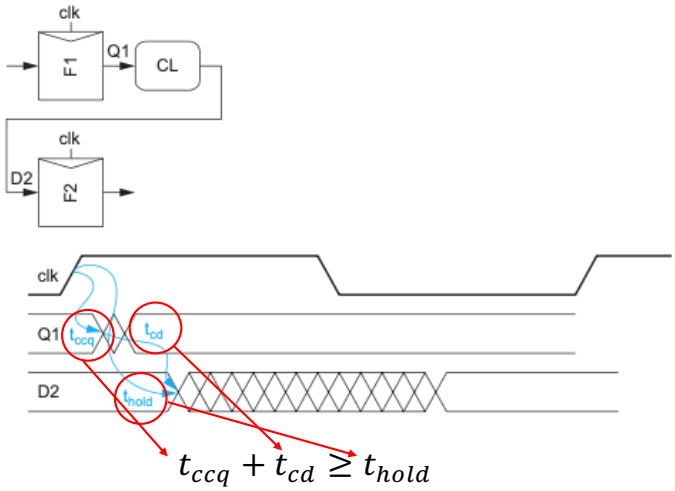
Combining these two cases gives:

$$T_c \geq \max\left(t_{pdq} + t_{pd},\ t_{pcq} + t_{pd} + t_{\text{setup}} - t_{pw}\right)$$

Solving for the maximum logic delay shows that the sequencing overhead is just one latch delay if the pulse is wide enough to hide the setup time.

$$t_{pd} \leq T_c - \underbrace{\max\left(t_{pdq},\ t_{pcq} + t_{\text{setup}} - t_{pw}\right)}_{\text{sequencing overhead}}$$

Flip-flop Minimum Delay Constraint



$$t_{ccq} + t_{cd} \geq t_{hold}$$

$$t_{cd} \geq t_{hold} - t_{ccq}$$

W&H p384-386

Minimum delay constraint is usually not an issue unless the hold time is large and the contamination delay is small. In that case, data can incorrectly propagate through two successive elements on one clock edge, corrupting the state of the system.

This is called a **race condition**, hold-time failure, or min-delay failure. Such timing problem CANNOT be solved by slowing the clock. It can only be fixed by redesigning the logic. Therefore, designers should be very conservative in avoiding such failures.

The diagram above shows the min-delay timing constraints on a path from one flip-flop to the next assuming ideal clocks with no skew. The path begins with the rising edge of the clock triggering F1. The data may begin to change at Q1 after a clk-to-Q contamination delay t_{ccq}, and at D2 after another logic contamination delay t_{cd}. However, it must not reach D2 until at least the hold time t_{hold} after the clock edge. Otherwise the contents of F2 will be corrupted.

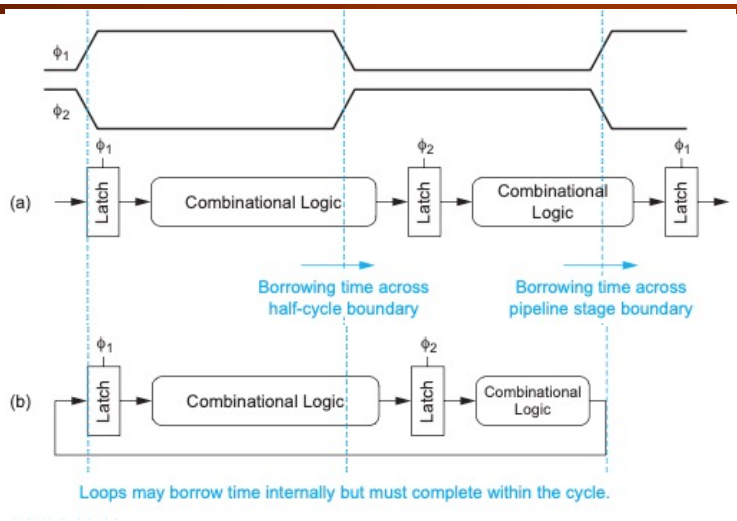
This provide us with the minimum delay to D2, the input of F2:

$$t_{cd} \geq t_{\text{hold}} - t_{ccq}$$

In most cases, the delay through the combinational logic and the contamination delay of D-FF is higher than the hold time. Such violation is therefore rare.

For minimum delay constraints for 2-phase clocking latch and for pulsed latch, see p. 384 to p.386 of Weste & Harris.

Time Borrowing (1)



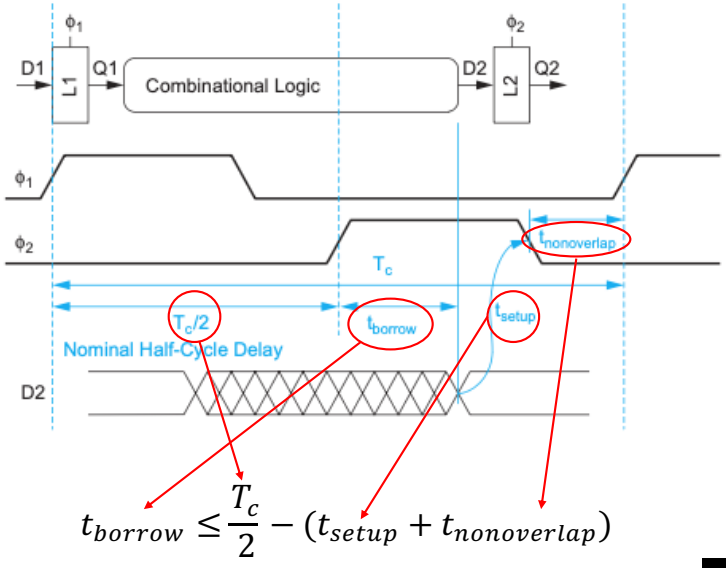
W&H p387

In a system using flip-flops, the clock imposes strict timing determined by the cycle time T_c because it sharply delineates the cycles.

In contrast, when a system uses transparent latches, the data can depart the first latch on the rising edge of the clock, but does not have to set up until the falling edge of the clock on the receiving latch. If one half-cycle or stage of a pipeline has too much logic, it can **borrow time** into the next half-cycle or stage, as illustrated in Figure (a) above.

Time borrowing can accumulate across multiple cycles. However, in systems with feedback, the long delays must be balanced by shorter delays so that the overall loop completes in the time available. For example, Figure 10.12(b) shows a single-cycle self-bypass loop in which time borrowing occurs across half-cycles, but the entire path must fit in one cycle.

Time Borrowing (2)



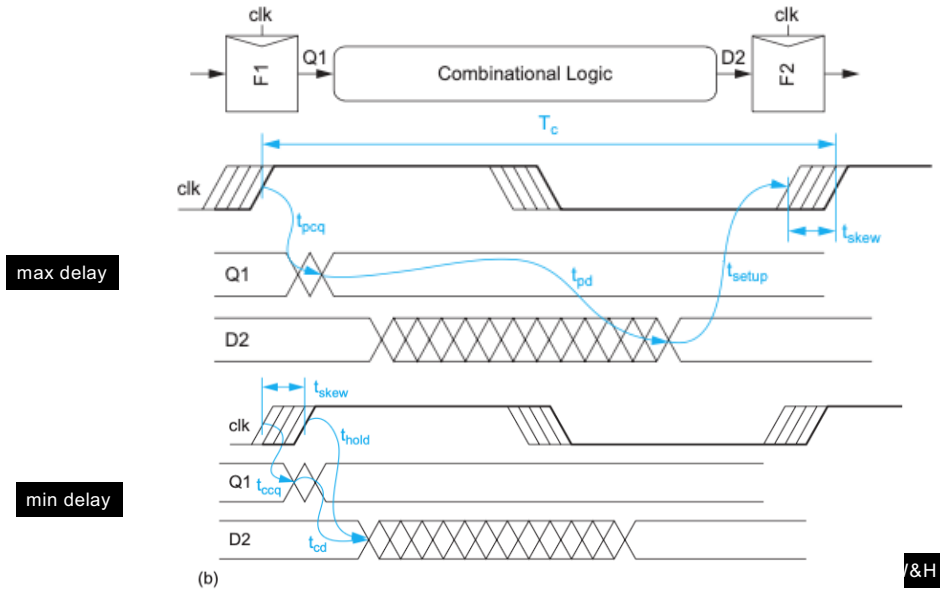
$$t_{borrow} \leq \frac{T_c}{2} - (t_{setup} + t_{nonoverlap})$$

W&H 387-389

Because data does not have to set up until the falling edge of the receiving latch's clock, one phase can borrow up to half a cycle of time from the next (less setup time and non-overlap), given by:

$$t_{\text{borrow}} \leq \frac{T_c}{2} - \left(t_{\text{setup}} + t_{\text{nonoverlap}}\right)$$

Clock Skew Problem in Flip-flop



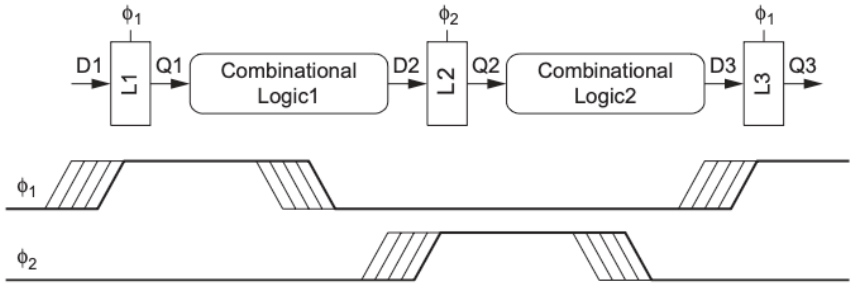
/&H 387-389

In real circuits, clock signals are not perfect. Clocks have some uncertainty in their arrival times that can cut into the time available for useful computation as shown above. The worst scenario for max delay in a flip-flop-based system is that the launching flop receives its clock late and the receiving flop receives its clock early.

In this case, the clock skew is subtracted from the time available for useful computation and appears as sequencing overhead. This is shown in the “max delay” timing above.

The worst scenario for min delay is that the launching flop receives its clock early and the receiving clock receives its clock late, as shown in Figure (b). In this case, the clock skew effectively increases the hold time of the system.

Clock Skew Problem in Transparent Latch



- Transparent latches are **skew tolerant**.

W&H 387-389

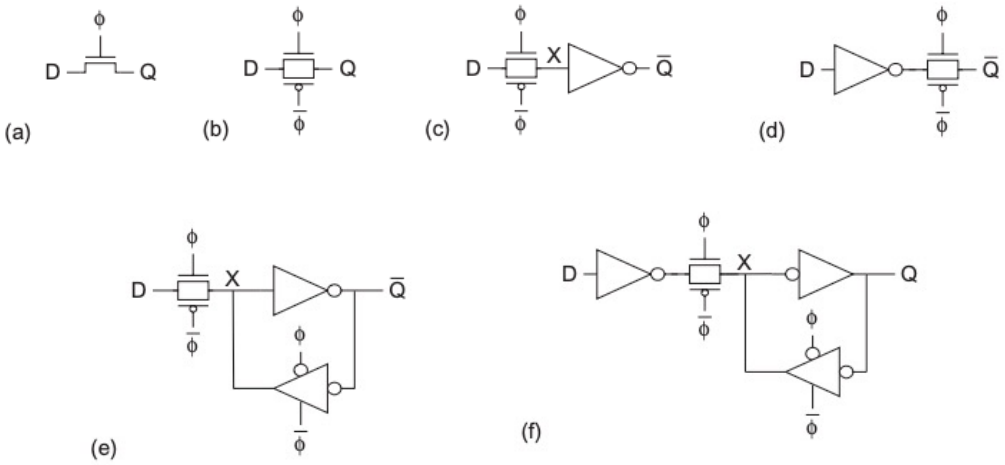
In the system using transparent latches, clock skew **does not degrade performance**.

As shown above, the full cycle (less two latch delays) is available for computation even when the clocks are skewed because the data can still arrive at the latches while they are transparent.

Therefore, we say that transparent latch-based systems are **skew-tolerant**.

However, skew still effectively increases the hold time in each half-cycle. It also cuts into the window available for time borrowing.

Transparent Latch Circuits (1)



W&H p392

(a) is the simplest latch – just a switch. It has many problems: 1) the output never rises above VDD – Vt; 2) the output floats when the latch is opaque; 3) it suffers from noise problem at both input and output.

(b) uses a CMOS transmission gate in place of the single nMOS, eliminating the VDD-Vt problem, but the other problems remain. It also requires complementary clock signals ϕ and $\bar{\phi}$.

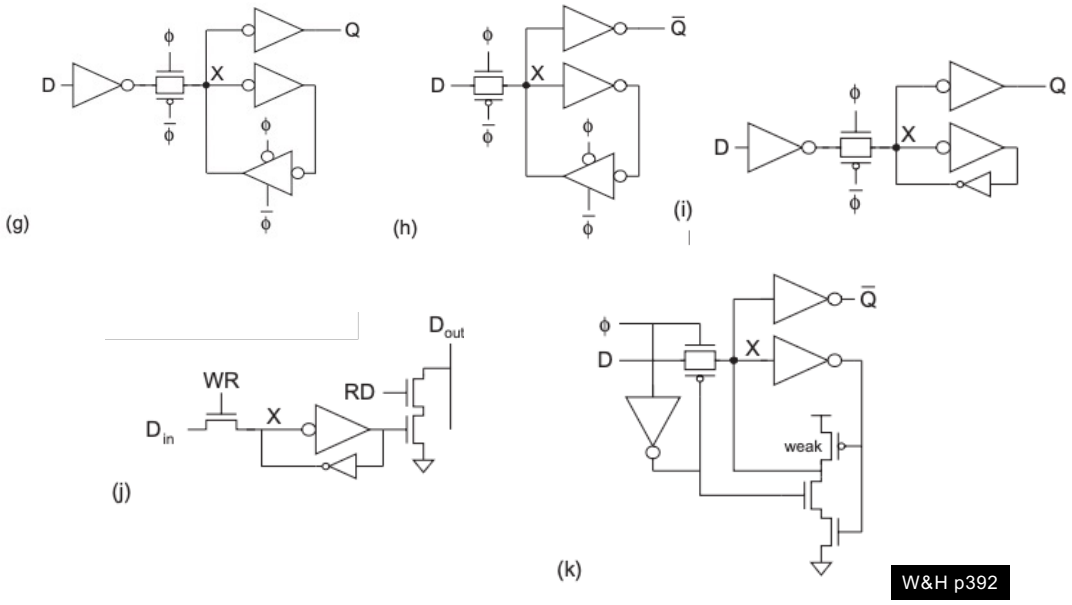
(c) adds an inverter which overcomes the floating output problem, but it still suffers from the leakage from the stored charge on X, and imposes a minimum clock frequency.

(d) is similar version of (c) with the transmission gate and inverter swapped. Both are fast dynamic latches.

(e) is a static version of (c). It avoid the subthreshold leakage problem with dynamic latches. Due to the feedback inverter, output is now never floating and the state storage node X is never floating. When the clock is '1' the latch samples the input. When the clock is 0, the transmission gate is off and the feedback inverter maintains the stored value.

(f) adds an input inverter so driving circuit is to transistor gates rather than unbuffered diffusion. Unfortunately, both (e) and (f) reintroduced output noise sensitivity: A large noise spike on the output can propagate backward through the feedback gates and corrupt the state node X.

Transparent Latch Circuits (2)



W&H p392

(g) is a robust transparent latch that addresses all of the deficiencies mentioned so far: the output noise sensitivity issue is overcome.

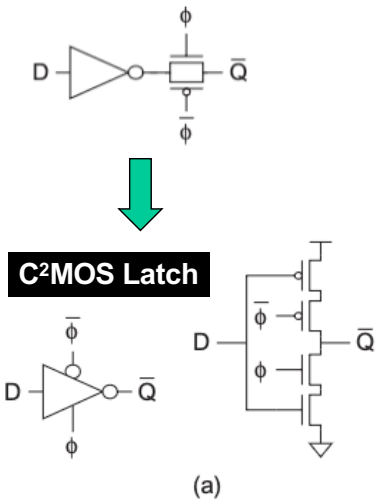
(h) is a reduced version of (g). It is faster, small and used where size matters and inverting latch is required.

(i) shows a practical design where the inverter to make the latch static is minimum size and eliminates the need for clock signals with the feedback latch. During sampling (clock = 1), the output of the first inverter will ”fight” the feedback inverter and WINS! During storage state, the stored value X is maintained by the feedback latch.

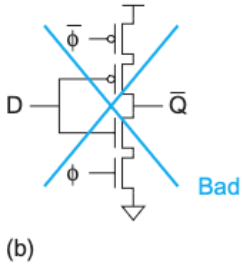
(j) shows a latch used in register files and FPGAs, where many latches share a Dout line, but only one latch is enabled at a time with the RD signal.

(k) is a latched used in Intel’s Itanium 2 processor. The pMOS is a weak transistor that provides a static pull-up at X. The pullup could fight with the full down asserted by D when the transmission gate is ON, but pMOS being a ”weak transistor”, X will take on the value of D. When clock = 0, the feedback inverter keeps X static.

C²MOS Latch Circuit



❑ Charge sharing problem.



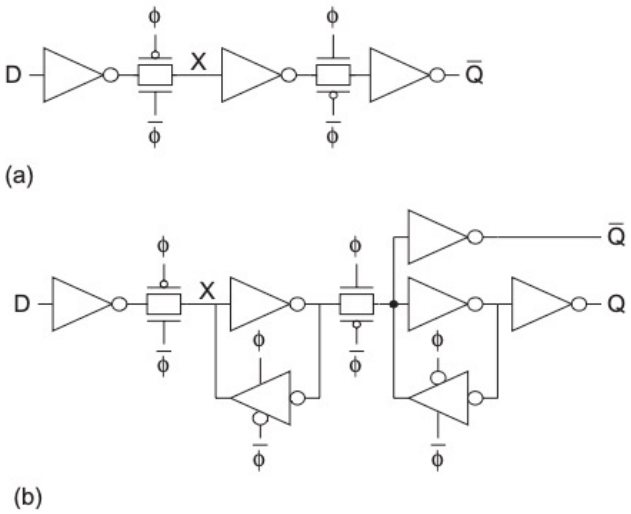
❑ Clocked CMOS Latch = a tri-state inverter

W&H p393

This is another simple latch called clocked CMOS or C^2MOS latch.

It is an implementation of a simple latch but with the transmission gate transistor "push" back into the output of the inverter. It is still a dynamic latch with the output $\bar{Q}$ being the storage node.

Simple Flip-flop – single phase clock



W&H p393

A flip-flop is implemented with two latches clocked by opposite phase of the clock.

(a) is a dynamic flip-flop.

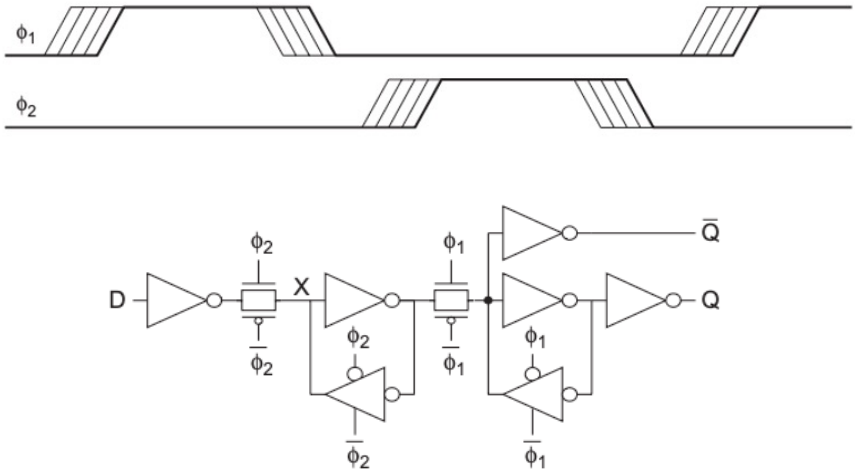
(b) is the static version of the dynamic flip-flop.

It is obvious that the flip-flop circuit is larger than the latch circuit.

Flip-flops usually take a single clock signal and locally generate its complement . If the clock rise/fall time is very slow, or with clock skew is serious, it is possible that both the clock and its complement will simultaneously be at intermediate voltages, making both latches transparent and increasing the flip-flop hold time.

In ASIC standard cell libraries, the clock is both complemented and buffered in the flip-flop cell to sharpen up the edge rates at the expense of more inverters and clock loading. This ensure robustness in the design but result in higher power consumption.

Two-phase clocking Flip-flop

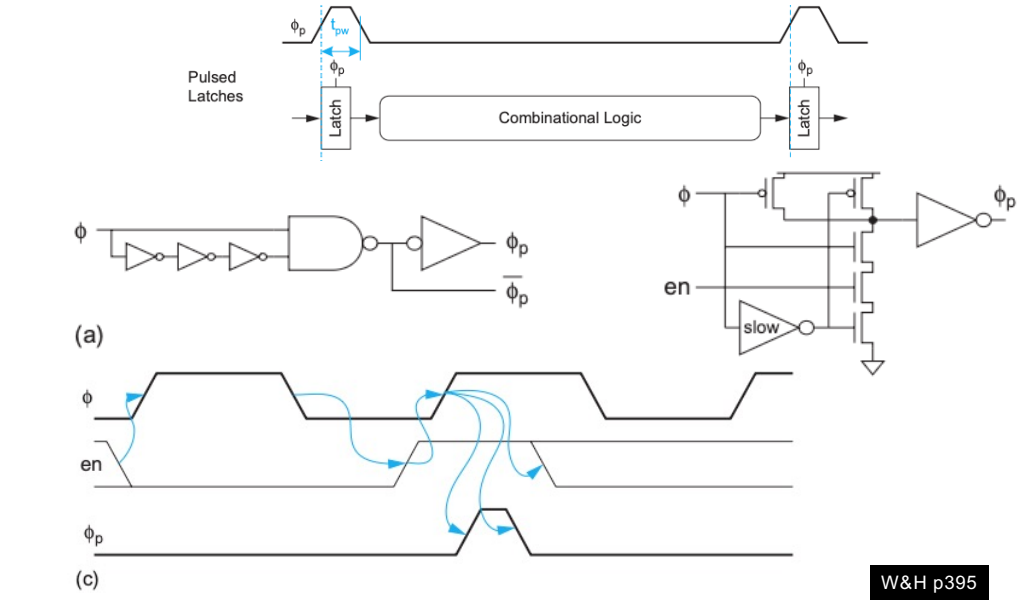


W&H p394

In designs where careful clock skew analysis is too much work and performance is less important, a reasonable alternative is to use a pair of two-phase non-overlapping clocks instead of the clock and its complement, as shown in the last slide.

The nonoverlap of the clock ensure that the circuit will work despite large skews. However, the nonoverlap time is not used by logic, so it directly increases the setup time and sequencing overhead of the flip-flop.

Pulse Generators

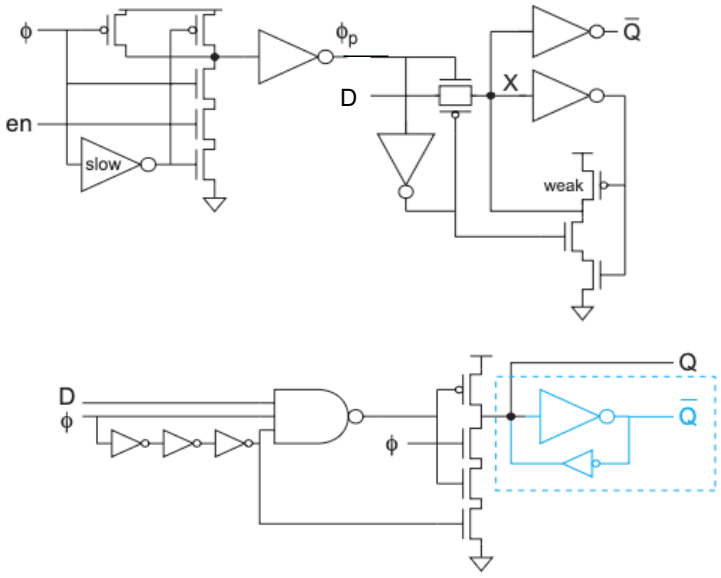


W&H p395

Instead of using a clock signal to control latches (and flip-flop), some modern design uses pulsed latches, which is faster and smaller than regular flip-flops or clocked latches.

The circuits above produces a short pulse on each rising clock edge. The width of the pulse is determined by the chain of inverters.

Pulse Latch Circuits

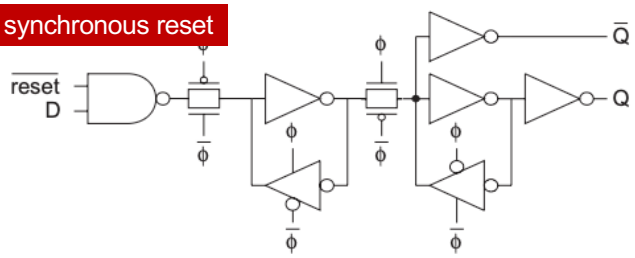


W&H p396

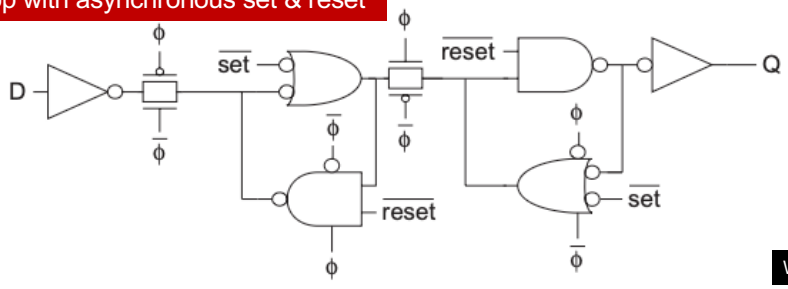
This slide shows the pulse latch circuit which is small and fast. A conventional latch is now driven by the pulse signal which samples the input D. It is then stored in the staticized latch and provide the state output.

Reset Flip-flop & Latches

Flip-flop with synchronous reset



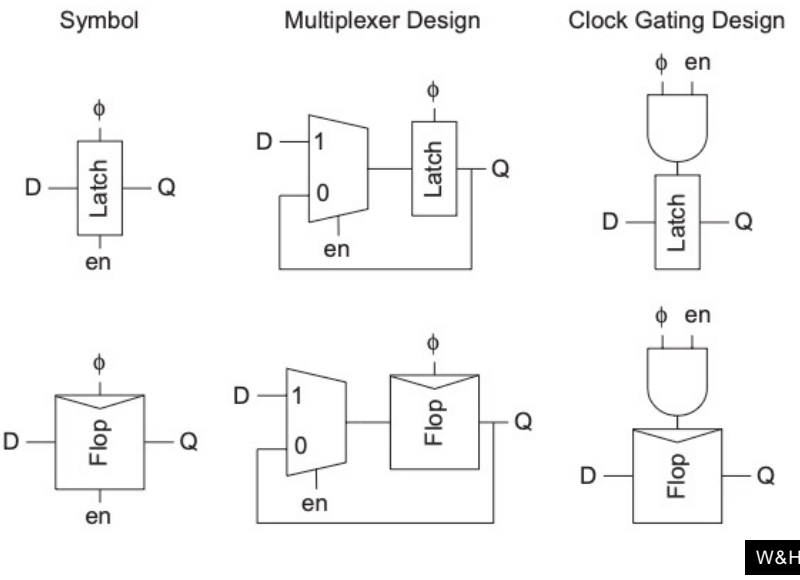
Flip-flop with asynchronous set & reset



W&H p397

This slide shows minor modification for flip-flop and latch circuits with reset signals. How it works is obvious.

Flip-flops & Latches with enable



W&H p398

Adding enable signal to flip-flop and latches is also simple.

One approach is to use a MUX to either select the input data D to be stored in the latch, or feed the output back to the input when the latch is NOT enabled.

Advantage of Latches Over Flip-Flops

- If you are going to use clk and clk_b and control skew, why not go back to flip-flops?
- Many people do:
  - Most designs in industry are based on flip-flops
  - Very easy to verify timing
  - Each path between flip-flops must be less than cycle time
  - Tools check for skew and hold time violations
  - Short paths are padded (buffers are added to slow down the signals)
  - Skew in flip-flop based systems affects the critical path
- Latch designs are more flexible than a flip-flop design
  - Gives the designer more rope
  - Need to CAD tools to make sure it works
  - Can borrow time to allow a path to be longer than clock period
  - Can tolerate clock skew -- skew does not directly add to cycle time

Here are list of reasons why latches are better than flip-flops in many VLSI circuits.

Disadvantages of two-phase clocking

- Need four clocks in general
  - Need true and complement of both clocks
- Still need low skew for good performance
  - The skew increases the cycle time of the machine
  - Need low skew between all the clocks for good performance
  - Want to have Φ1 and Φ2 close to coincident
- Many systems use clock and its complement instead of 2 phases
  - Needless to say they are very careful about clock skew
  - For these systems it is still useful to maintain 2 phase timing types, since it ensures you connect all logic to the right latches
  - Call Clk - Φ1 and Clk - Φ2, and go from there.

Here is a list of disadvantages of using two-phase clocking latches and flip-flops.

It brings the question: can we design a latch (or flip-flop) that only uses a single clock signal (without its complement)?